Cloud Computing

[FINAL PROJECT]

High-Performance Web Hosting Platform

Submitted By:

- Manahil Habib(2023-BSE-042)
- Maria Irfan(2023-BSE-044)

Section:

BSE-V-B

Submitted to:

Sir Waqas

Date of Submission

Jan 27, 2026

1. Abstract

This project demonstrates the design and deployment of a **high-performance, production-style web hosting platform** built entirely on **Amazon Web Services (AWS)**. The infrastructure is provisioned using **Terraform**, while server configuration and application deployment are automated using **Ansible**.

The platform is capable of hosting **multiple customer websites** on shared infrastructure using **Nginx virtual hosts**, with **PHP-FPM** handling dynamic PHP content. Instead of using AWS-managed services like Application Load Balancer (ALB), this project implements a **self-managed Nginx-based load balancing layer**, providing deeper understanding of reverse proxy architecture and traffic distribution.

The solution emphasizes:

- High availability through **Multi-AZ deployment**
- Infrastructure automation via **Infrastructure as Code**
- Secure architecture using **public/private subnet separation**
- Multi-tenant hosting using **server blocks**
- Performance optimization for **Nginx and PHP-FPM**
- Automated customer onboarding
- Monitoring using custom **Bash scripts**

This project reflects real-world cloud hosting practices used by modern infrastructure and DevOps teams.

2. Introduction

Web hosting providers must deliver platforms that are **scalable, secure, automated, and highly available**. Traditional manual server setup is slow, error-prone, and difficult to scale. Modern cloud engineering relies on:

- **Infrastructure as Code (IaC)** for provisioning
- **Configuration Management** for software setup
- **Load balancing** for traffic distribution

- **Automation** for repeatability

This project simulates a **mini cloud hosting provider** using only tools covered in the course. It demonstrates how a multi-tenant hosting platform can be built using **AWS primitives and open-source technologies**.

3. System Architecture

3.1 Architectural Goals

The architecture was designed to achieve:

- High availability across Availability Zones
- Secure isolation of backend servers
- Support for multiple customer websites
- Load distribution without AWS-managed LB
- Easy scaling and repeatable deployments

3.2 Tiered Architecture

The system is divided into **two logical tiers** inside a custom AWS VPC.

1️⃣ Load Balancer Tier (Public Subnets)

This tier contains EC2 instances running **Nginx as a reverse proxy and load balancer**.

Responsibilities:

- Accept incoming HTTP/HTTPS requests from users
- Terminate SSL connections
- Distribute traffic across backend web servers
- Cache static responses to improve performance

These instances are placed in **public subnets** so they can receive internet traffic.

2️⃣ Web Server Tier (Private Subnets)

This tier hosts EC2 instances running:

- Nginx (web server)
- PHP-FPM (PHP processing engine)

Responsibilities:

- Serve static files
- Process dynamic PHP content
- Host multiple websites using server blocks

These servers are in **private subnets** and **cannot be accessed directly from the internet**, improving security.

3.3 Traffic Flow

Client Browser → Internet → Nginx Load Balancer → Backend Web Servers → PHP-FPM → Response to Client

4. Git Repository & DevOps Workflow

A structured GitHub repository was created to maintain clean project organization and version control.

4.1 Repository Structure

Directory	Purpose
terraform/	Infrastructure provisioning
ansible/	Server configuration
websites/	Customer website files
docs/	Documentation

This separation supports **modularity and reusability**.

Install Required Tools

```
@mariahirfan658-ui ➜/workspaces/final-proj (main) $ terraform -v
ansible --version
aws --version
git --version
Terraform v1.14.3
on linux_amd64
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/codespace/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/codespace/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Nov  6 2025, 13:44:16) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.6
  libyaml = True
aws-cli/2.33.6 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
git version 2.52.0
@mariahirfan658-ui ➜/workspaces/final-proj (main) $
```

Configure AWS Credentials

```
@mariahirfan658-ui ➜/workspaces/CC-final-proj (main) $ aws configure
AWS Access Key ID [None]: A[redacted]
AWS Secret Access Key [None]: [redacted]
Default region name [None]: me-central-1
Default output format [None]: json
@mariahirfan658-ui ➜/workspaces/CC-final-proj (main) $
```

Create Project Structure

```
@mariahirfan658-ui ➜/workspaces/final-proj (main) $ mkdir -p Project6/{terraform/{environments,modules/{network,lb-ec2,web-ec2}},ansible/
{inventory,playbooks,roles/{web-stack,lb-nginx,websites,monitoring}},websites/{customer1,customer2,default},docs}
@mariahirfan658-ui ➜/workspaces/final-proj (main) $ touch Project6/README.md Project6/.gitignore
@mariahirfan658-ui ➜/workspaces/final-proj (main) $
@mariahirfan658-ui ➜/workspaces/final-proj (main) $ cd Project6
```

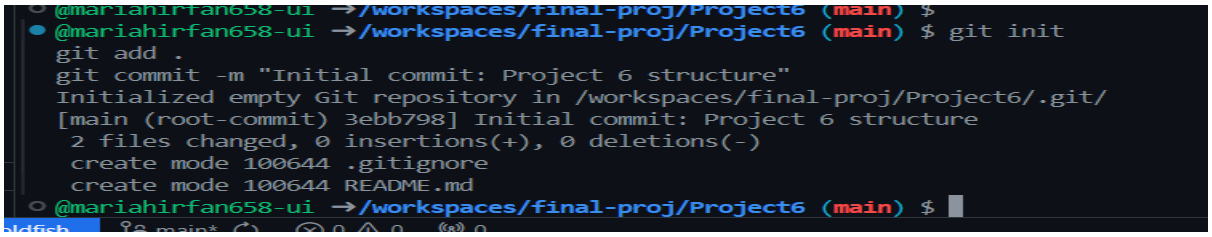
4.2 Branching Strategy

A professional Git branching model was followed:

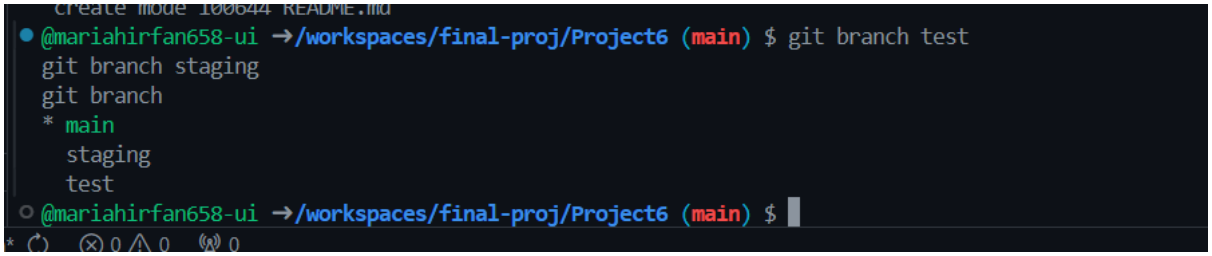
Branch	Environment
main	Production
staging	Pre-production
test	Testing
feature/*	New features

This approach simulates a **CI/CD-ready workflow**.

Initialize Git



Create branches:



4.3 Security with .gitignore

Sensitive files were excluded:

- Terraform state files
- AWS credentials
- SSH private keys
- Logs and temporary files

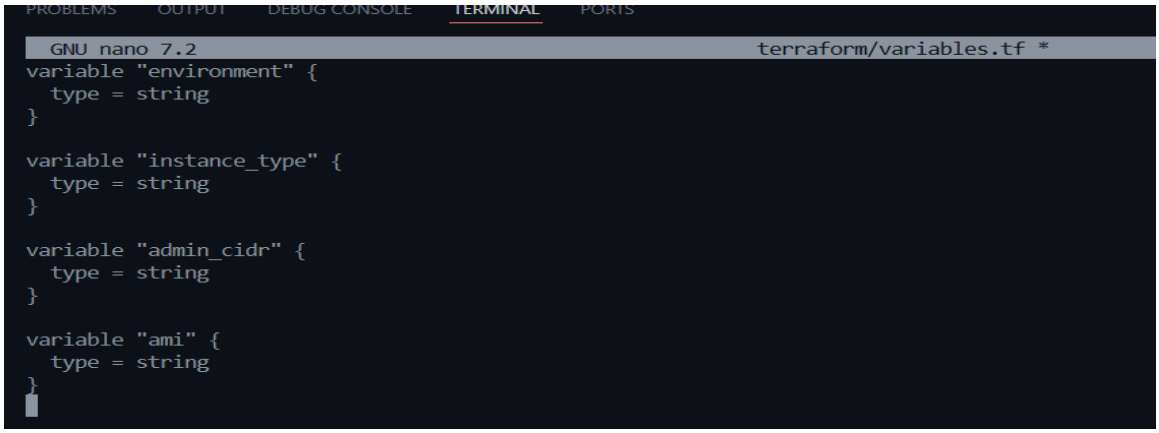
This ensures the repository is **safe for public sharing**.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

 GNU nano 7.2                                                .gitignore *

# AWS credentials
.aws/
*.pem
*.key

# IDE
.vscode/
.idea/
*.swp
*.swo
*~

# OS
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Environment files
.env
.env.local

# Temp
tmp/
temp/
```

5. Terraform Infrastructure Implementation

Terraform was used to provision all AWS resources in a **modular and reusable manner**.

```
 GNU nano 7.2                                          terraform/main.tf *
provider "aws" {
  region = "me-central-1"
}

module "network" {
  source = "./modules/network"

  vpc_cidr        = "10.0.0.0/16"
  public_subnets  = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24"]
  azs             = ["me-central-1a", "me-central-1b"]
  env             = var.environment
}
```



5.1 Network Module

A custom VPC was created to isolate the hosting platform.

Component	Description
VPC	10.0.0.0/16
Public Subnets	For Load Balancers
Private Subnets	For Web Servers
Internet Gateway	Internet access for LB
NAT Gateway	Allows private servers to download updates

This design follows **best practices for secure cloud networking**.

```
 GNU nano 7.2                                        terraform/modules/network/main.tf *
esource "aws_vpc" "this" {
 cidr_block = var.vpc_cidr
 enable_dns_support   = true
 enable_dns_hostnames = true

 tags = {
   Name = "${var.env}-vpc"
   Project = "Project6"
 }


esource "aws_internet_gateway" "igw" {
 vpc_id = aws_vpc.this.id
 tags = { Name = "${var.env}-igw" }


esource "aws_subnet" "public" {
 count                   = 2
 vpc_id                  = aws_vpc.this.id
 cidr_block              = var.public_subnets[count.index]
 availability_zone       = var.azs[count.index]
 map_public_ip_on_launch = true

 tags = {
   Name = "${var.env}-public-${count.index + 1}"
   Tier = "public"
 }


G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Location
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
 GNU nano 7.2                                        terraform/modules/network/outputs.tf *
output "vpc_id" {
  value = aws_vpc.this.id
}

output "public_subnets" {
  value = aws_subnet.public[*].id
}

output "private_subnets" {
  value = aws_subnet.private[*].id
}
```

```
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $ terraform init
Initializing the backend...
Initializing modules...
- network in modules/network
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $ terraform apply -var-file=environments/test.tf
```

```
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $ terraform apply -var-file=environments/test.tfvars -auto-approve
module.network.aws_route_table.public_rt: Creating...
module.network.aws_subnet.private[0]: Creation complete after 1s [id=subnet-0b1cef15213453879]
module.network.aws_route_table.public_rt: Creation complete after 1s [id=rtb-09608c3af43f77d95]
module.network.aws_subnet.private[1]: Creation complete after 4s [id=subnet-0b84db82ab2f4dcc8]
module.network.aws_subnet.public[0]: Still creating... [00m10s elapsed]
module.network.aws_subnet.public[1]: Still creating... [00m10s elapsed]
module.network.aws_subnet.public[1]: Creation complete after 11s [id=subnet-07483eb9e21da628c]
module.network.aws_subnet.public[0]: Creation complete after 11s [id=subnet-08f71552c726e59fb]
module.network.aws_route_table_association.public_assoc[0]: Creating...
module.network.aws_route_table_association.public_assoc[1]: Creating...
module.network.aws_nat_gateway.nat: Creating...
module.network.aws_route_table_association.public_assoc[0]: Creation complete after 1s [id=rtbassoc-09fd2e61e67a635bf]
module.network.aws_route_table_association.public_assoc[1]: Creation complete after 1s [id=rtbassoc-0dbde8baf01ed3005]
module.network.aws_nat_gateway.nat: Still creating... [00m10s elapsed]
module.network.aws_nat_gateway.nat: Still creating... [00m20s elapsed]
module.network.aws_nat_gateway.nat: Still creating... [00m30s elapsed]
module.network.aws_nat_gateway.nat: Still creating... [00m40s elapsed]
module.network.aws_nat_gateway.nat: Still creating... [00m50s elapsed]
module.network.aws_nat_gateway.nat: Still creating... [01m00s elapsed]
module.network.aws_nat_gateway.nat: Still creating... [01m10s elapsed]
module.network.aws_nat_gateway.nat: Still creating... [01m20s elapsed]
module.network.aws_nat_gateway.nat: Creation complete after 1m25s [id=nat-00d8a71fc5ed72326]
module.network.aws_route_table.private_rt: Creating...
module.network.aws_route_table.private_rt: Creation complete after 1s [id=rtb-0f29b2d3833e76a98]
module.network.aws_route_table_association.private_assoc[1]: Creating...
module.network.aws_route_table_association.private_assoc[0]: Creating...
module.network.aws_route_table_association.private_assoc[1]: Creation complete after 0s [id=rtbassoc-0e21c8dc8118fe905]
module.network.aws_route_table_association.private_assoc[0]: Creation complete after 0s [id=rtbassoc-038552666c2df4584]

Apply complete! Resources: 14 added, 0 changed, 0 destroyed.
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $
```

5.2 Load Balancer EC2 Module

Two EC2 instances were provisioned across different AZs.

Security rules:

- Allow HTTP (80) and HTTPS (443) from anywhere
- Allow SSH only from admin IP

These instances form the **entry point of the hosting platform**.

```
 GNU nano 7.2                                    modules/sg/main.tf *
variable "vpc_id" {}
variable "admin_cidr" {}

resource "aws_security_group" "lb_sg" {
  name        = "lb-sg"
  description = "Allow HTTP + SSH from admin"
  vpc_id      = var.vpc_id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.admin_cidr]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
 GNU nano 7.2                                    modules/ec2/main.tf *
variable "ami" {}
variable "instance_type" {}
variable "subnets" { type = list(string) }
variable "sg_id" {}
variable "name" {}
variable "count" { default = 1 }
variable "key_name" {}

resource "aws_instance" "this" {
  count                       = var.count
  ami                         = var.ami
  instance_type               = var.instance_type
  subnet_id                   = var.subnets[count.index % length(var.subnets)]
  vpc_security_group_ids      = [var.sg_id]
  key_name                    = var.key_name
  associate_public_ip_address = true

  tags = {
    Name = "${var.name}-${count.index + 1}"
  }
}

output "instance_ids" { value = aws_instance.this[*].id }
output "private_ips"  { value = aws_instance.this[*].private_ip }
output "public_ips"   { value = aws_instance.this[*].public_ip }
```

[Preview] README.md | main.tf

Project6 > terraform > main.tf

```
}

module "sg" {
  source     = "./modules/sg"
  vpc_id     = module.network.vpc_id
  admin_cidr = var.admin_cidr
}

module "lb" {
  source        = "./modules/ec2"
  name          = "lb"
  ami           = var.ami
  instance_type = var.instance_type
  subnets       = module.network.public_subnet_ids
  sg_id         = module.sg.lb_sg_id
  key_name      = var.key_name
}

module "web" {
  source        = "./modules/ec2"
  name          = "web"
  count         = 2
  ami           = var.ami
  instance_type = var.instance_type
  subnets       = module.network.private_subnet_ids
  sg_id         = module.sg.web_sg_id
  key_name      = var.key_name
}

```

[Preview] README.md | main.tf | variables.tf

roject6 > terraform > variables.tf

```
variable "admin_cidr" {
}

variable "ami" {
  type = string
}
variable "key_name" {
  description = "EC2 SSH key pair name"
  type        = string
}

```

```
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $ terraform init
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $
```

```
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $ terraform init
terraform apply -var-file=environments/test.tfvars -auto-approve
          + "Name" = "web-sg"
        }
      + vpc_id                 = "vpc-09eda43d12d848b0e"
    }

Plan: 2 to add, 0 to change, 0 to destroy.
module.sg.aws_security_group.lb_sg: Creating...
module.sg.aws_security_group.lb_sg: Creation complete after 3s [id=sg-0aff4ce59b6c60e76]
module.sg.aws_security_group.web_sg: Creating...
module.sg.aws_security_group.web_sg: Creation complete after 4s [id=sg-019d6fe9ae62869b4]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $
```

5.3 Web Server EC2 Module

Backend web servers were deployed in private subnets.

Security rules:

- Allow HTTP only from LB security group
- SSH restricted to admin IP

This ensures backend servers are **not publicly exposed**.

```
  GNU nano 7.2                                                modules/ec2/main.tf *
resource "aws_instance" "this" {
  count         = var.count
  ami           = var.ami
  instance_type = var.instance_type
  subnet_id     = element(var.subnets, count.index)
  vpc_security_group_ids = [var.sg_id]
  key_name      = var.key_name

  tags = {
    Name = "${var.name}-${count.index + 1}"
    Role = var.name
  }
}
```

```
  GNU nano 7.2                                                modules/ec2/outputs.tf *
output "instance_ids" {
  value = aws_instance.this[*].id
}

output "private_ips" {
  value = aws_instance.this[*].private_ip
}

output "public_ips" {
  value = aws_instance.this[*].public_ip
}
```

```
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $ terraform init
terraform apply -var-file=environments/test.tfvars -auto-approve

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

Plan: 3 to add, 0 to change, 0 to destroy.
module.lb.aws_instance.this: Creating...
module.web[1].aws_instance.this: Creating...
module.web[0].aws_instance.this: Creating...
module.lb.aws_instance.this: Still creating... [00m10s elapsed]
module.web[1].aws_instance.this: Still creating... [00m10s elapsed]
module.web[0].aws_instance.this: Still creating... [00m10s elapsed]
module.web[1].aws_instance.this: Creation complete after 13s [id=i-0652a519ebc61efb1]
module.web[0].aws_instance.this: Creation complete after 14s [id=i-0e63d6333a5f81446]
module.lb.aws_instance.this: Creation complete after 14s [id=i-024d9f0551f3c027c]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

terraform output and EC2 Ips:

```
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $ aws ec2 describe-instances --region me-central-1 \
  --query "Reservations[].Instances[].PublicIpAddress" --output table
-------------------
| DescribeInstances|
+------------------+
|  158.252.72.125  |
|  3.28.40.136     |
|  158.252.93.228  |
|  158.252.32.142  |
+------------------+
@mariahirfan658-ui →/workspaces/final-proj/Project6/terraform (main) $
```

6. Environment-Based Deployments

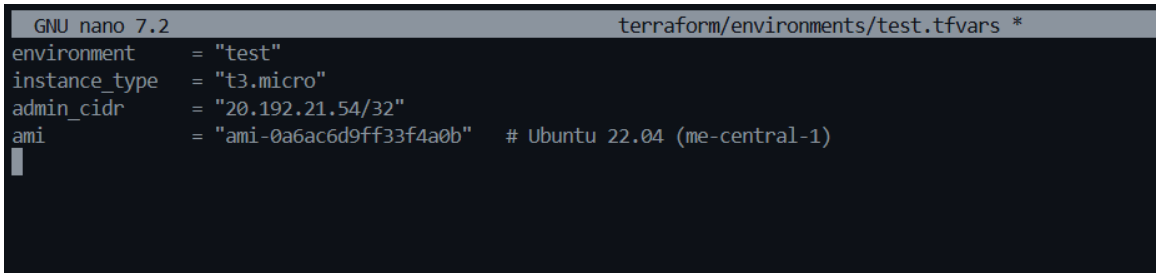
Terraform variable files allow different configurations for:

Environment	Purpose
test	Development testing
staging	Pre-production validation
production	Final deployment

Each environment can define:

- Instance types
- Number of servers
- Allowed IP ranges

This supports **scalable infrastructure management**.



7. Ansible Configuration Management

Ansible was used to configure all servers after provisioning.

7.1 Web Stack Role

This role prepares backend web servers.

Installed:

- Nginx
- PHP-FPM

Nginx Performance Tuning

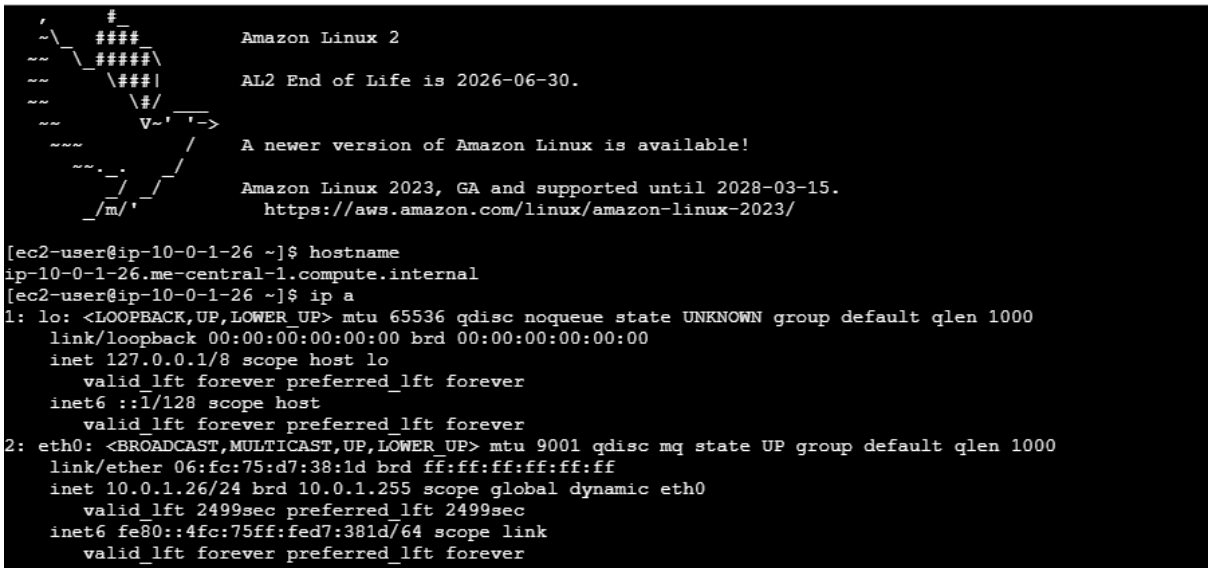
- worker_processes auto;
- worker_connections 1024;
- keepalive_timeout 65;
- gzip on;

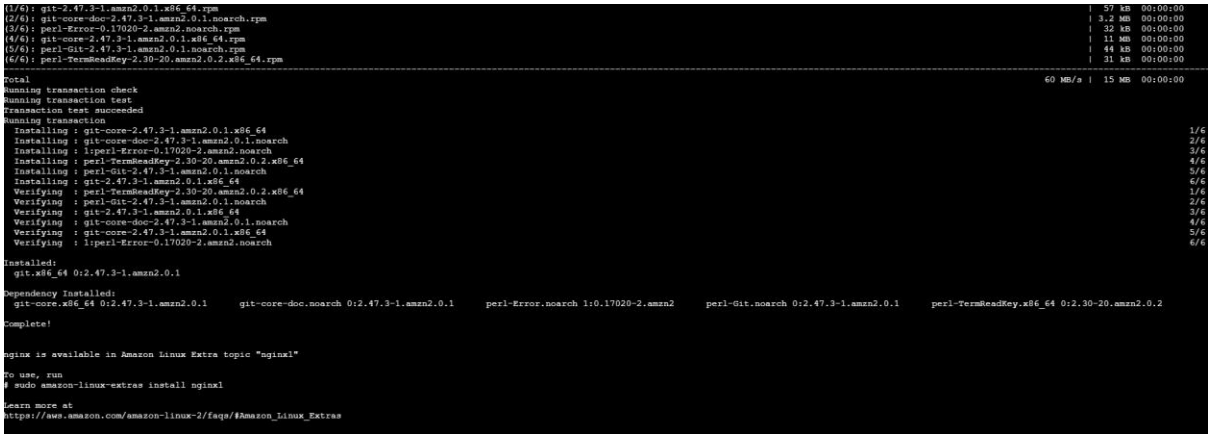
PHP-FPM Optimization

- pm = dynamic
- pm.max_children
- pm.start_servers

These settings improve **concurrency and response times**.

Ansible Inventory

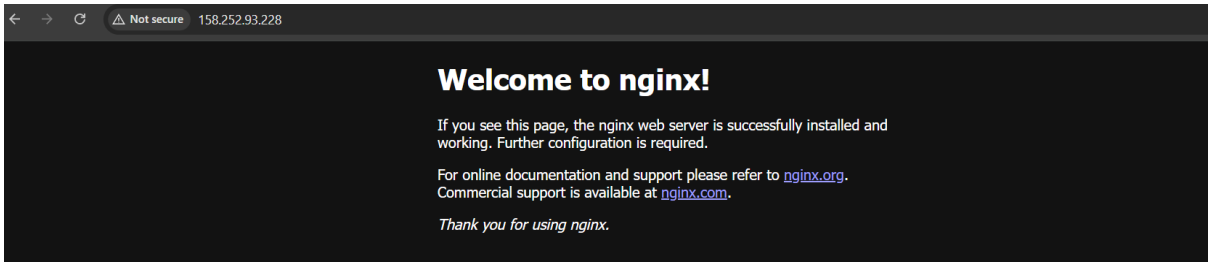




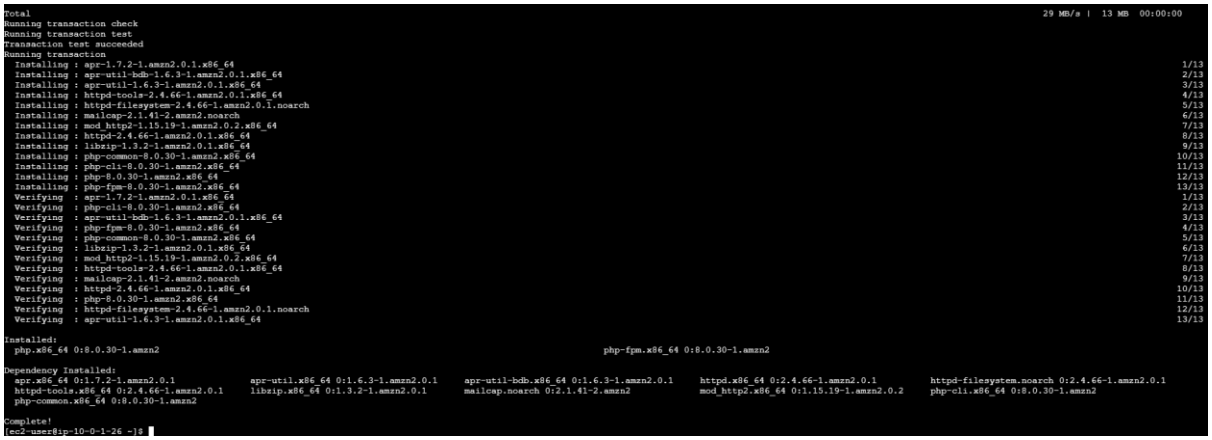
Nginx Install

```
Install  1 Package (+6 Dependent packages)

Total download size: 2.5 M
Installed size: 6.9 M
Downloading packages:
(1/7): nginx-1.28.1-1.amzn2.0.1.x86_64.rpm
(2/7): gperftools-libs-2.6.1-1.amzn2.x86_64.rpm
(3/7): generic-logos-httpd-18.0.0-4.amzn2.noarch.rpm
(4/7): nginx-filesystem-1.28.1-1.amzn2.0.1.noarch.rpm
(5/7): nginx-core-1.28.1-1.amzn2.0.1.x86_64.rpm
(6/7): openssl11-libs-1.1.1zd-1.amzn2.0.1.x86_64.rpm
(7/7): openssl11-pkcs11-0.4.10-6.amzn2.0.1.x86_64.rpm
--------------------------------------------------------------------------------------------------------------------------------------------------------
Total
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : 1:nginx-filesystem-1.28.1-1.amzn2.0.1.noarch
  Installing : 1:openssl11-libs-1.1.1zd-1.amzn2.0.1.x86_64
  Installing : openssl11-pkcs11-0.4.10-6.amzn2.0.1.x86_64
  Installing : generic-logos-httpd-18.0.0-4.amzn2.noarch
  Installing : gperftools-libs-2.6.1-1.amzn2.x86_64
  Installing : 1:nginx-core-1.28.1-1.amzn2.0.1.x86_64
  Installing : 1:nginx-1.28.1-1.amzn2.0.1.x86_64
  Verifying  : gperftools-libs-2.6.1-1.amzn2.x86_64
  Verifying  : openssl11-pkcs11-0.4.10-6.amzn2.0.1.x86_64
  Verifying  : 1:nginx-filesystem-1.28.1-1.amzn2.0.1.noarch
  Verifying  : 1:nginx-1.28.1-1.amzn2.0.1.x86_64
  Verifying  : 1:nginx-core-1.28.1-1.amzn2.0.1.x86_64
  Verifying  : generic-logos-httpd-18.0.0-4.amzn2.noarch
  Verifying  : 1:openssl11-libs-1.1.1zd-1.amzn2.0.1.x86_64

Installed:
  nginx.x86_64 1:1.28.1-1.amzn2.0.1

Dependency Installed:
  generic-logos-httpd.noarch 0:18.0.0-4.amzn2    gperftools-libs.x86_64 0:2.6.1-1.amzn2    nginx-core.x86_64 1:1.28.1-1.amzn2.0.1    nginx-filesystem.noarch
  openssl11-pkcs11.x86_64 0:0.4.10-6.amzn2.0.1

Complete!
[ec2-user@ip-10-0-1-26 ~]$
```

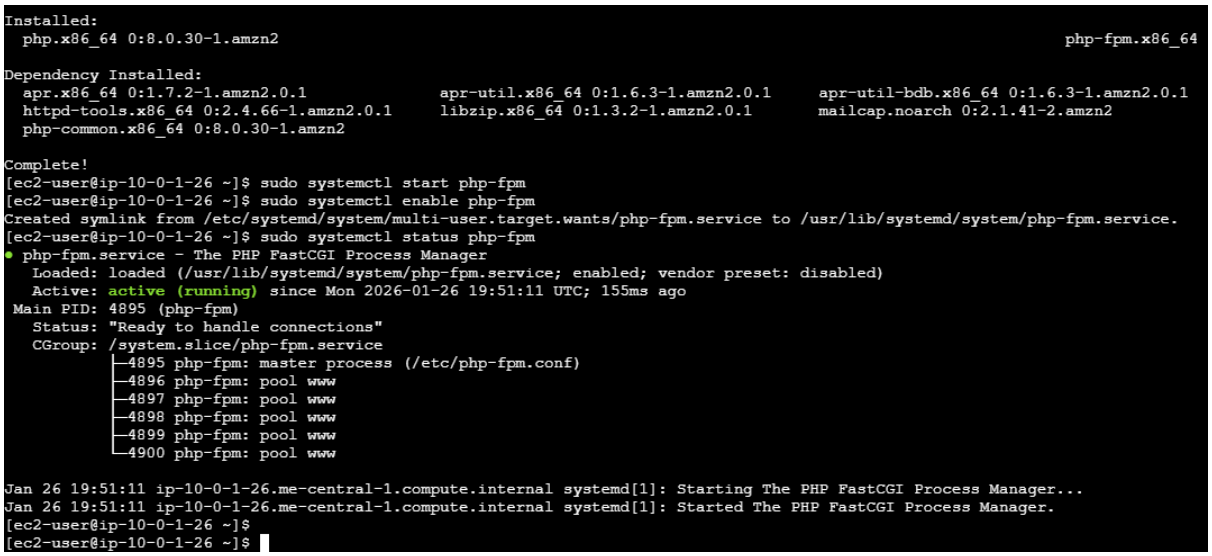
```
[ec2-user@ip-10-0-1-26 ~]$ sudo systemctl start nginx
[ec2-user@ip-10-0-1-26 ~]$ sudo systemctl enable nginx
Created symlink from /etc/systemd/system/multi-user.target.wants/nginx.service to /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-10-0-1-26 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2026-01-26 19:46:33 UTC; 39s ago
 Main PID: 4680 (nginx)
   CGroup: /system.slice/nginx.service
           ├─4680 nginx: master process /usr/sbin/nginx
           ├─4681 nginx: worker process
           └─4682 nginx: worker process

Jan 26 19:46:33 ip-10-0-1-26.me-central-1.compute.internal systemd[1]: Starting The nginx HTTP and reverse proxy server...
Jan 26 19:46:33 ip-10-0-1-26.me-central-1.compute.internal nginx[4675]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Jan 26 19:46:33 ip-10-0-1-26.me-central-1.compute.internal nginx[4675]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Jan 26 19:46:33 ip-10-0-1-26.me-central-1.compute.internal systemd[1]: Started The nginx HTTP and reverse proxy server.
[ec2-user@ip-10-0-1-26 ~]$
```



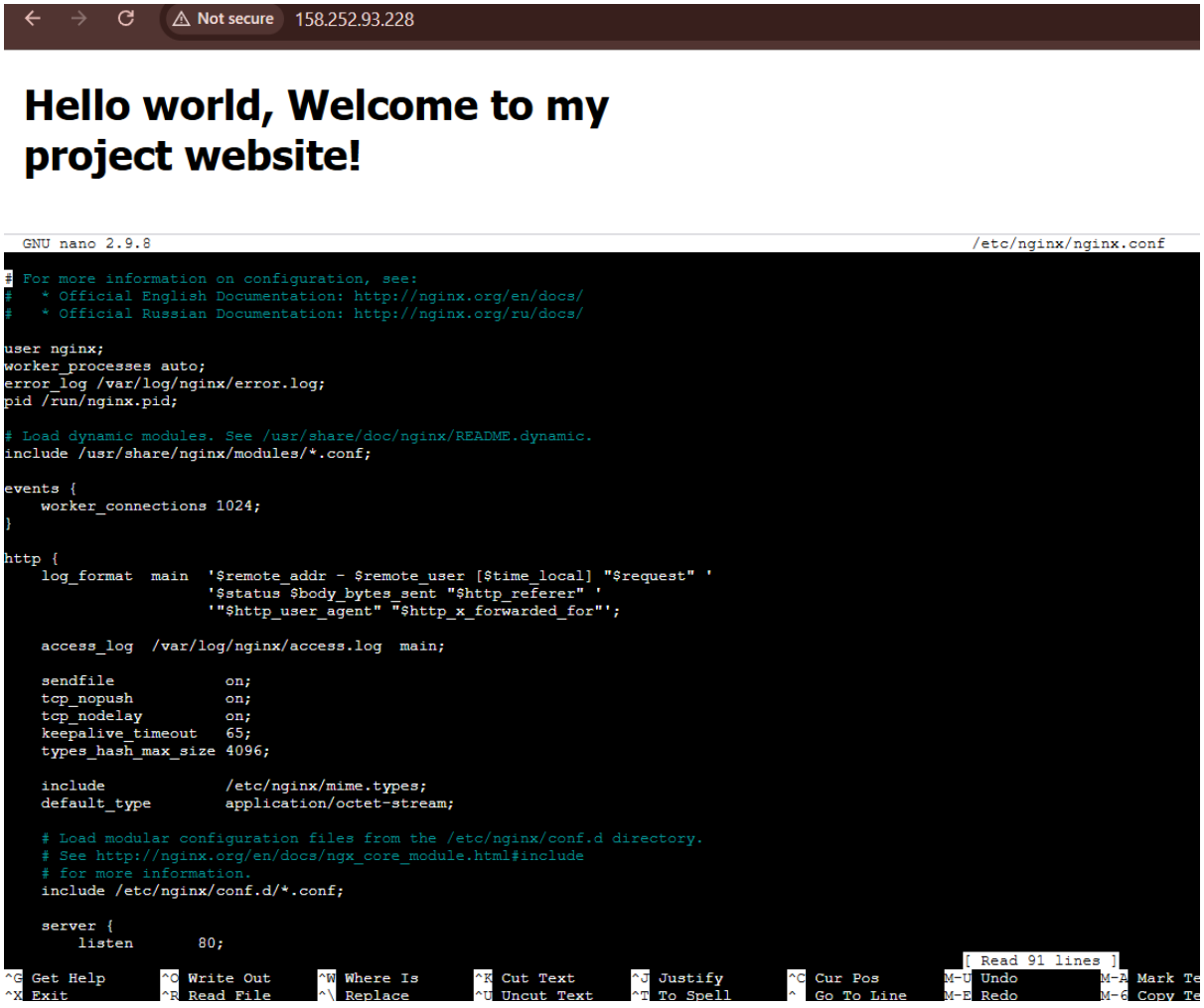
PHP-FPM Install



PHP-FPM Start + Enable



PHP Test File



7.2 Multi-Tenant Hosting

Each customer site has:

- Separate directory under /var/www/
- Separate Nginx server block
- Independent logs

This enables **shared hosting architecture**.

7.3 Load Balancer Configuration

The LB servers run Nginx configured with:

- upstream block for backend servers
- SSL termination

- HTTP → HTTPS redirection
- Proxy caching
- Backup server support

Traffic is automatically routed to healthy backend servers.

7.4 SSL/TLS Setup

Self-signed certificates were generated using OpenSSL.

Benefits:

- Encrypted communication
- HTTPS enforced
- Realistic hosting provider behavior

7.5 Automated Website Onboarding

A new website can be deployed using:

ansible-playbook add-website.yml -e "site_name=client3 server_name=client3.local"

Automation includes:

- Directory creation
- Server block generation
- File deployment
- Nginx reload

This simulates **real hosting automation**.

```
@mariahirfan658-ui ➜/workspaces/final-proj/Project6/terraform (main) $ ansible we
b -i inventory.ini -m ping
[WARNING]: Platform linux on host 3.29.137.251 is using the discovered Python
interpreter at /usr/bin/python3.7, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
3.29.137.251 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.7"
    },
    "changed": false,
    "ping": "pong"
}
@mariahirfan658-ui ➜/workspaces/final-proj/Project6/terraform (main) $
```

```
@mariahirfan658-ui ➜/workspaces/final-proj/Project6/terraform (ma
in) $ ansible-playbook -i inventory.ini web.yml

TASK [Gathering Facts] *********************************************************
[WARNING]: Platform linux on host 3.29.137.251 is using the discovered Python
interpreter at /usr/bin/python3.7, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [3.29.137.251]

TASK [Install apache] **********************************************************
ok: [3.29.137.251]

TASK [Start apache] ************************************************************
ok: [3.29.137.251]

PLAY RECAP *********************************************************************
3.29.137.251               : ok=3    changed=0    unreachable=0    failed=0    skipp
ed=0    rescued=0    ignored=0
@mariahirfan658-ui ➜/workspaces/final-proj/Project6/terraform (main) $
```

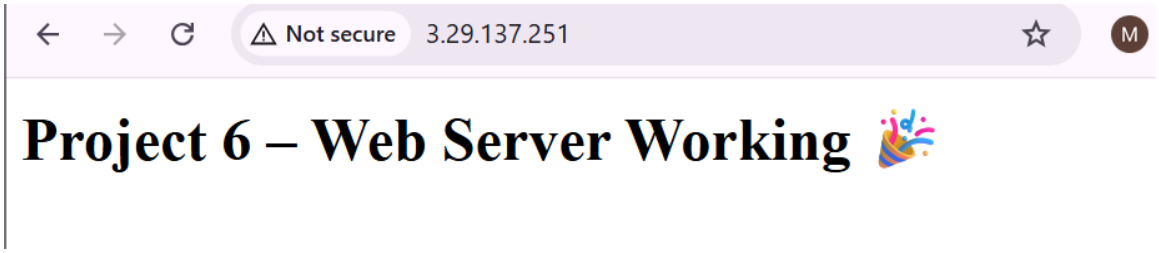
Web Server from ip-10-0-1-59.me-central-1.compute.internal

```
@mariahirfan658-ui ➜/workspaces/final-proj/Project6/terraform (main) $ ansible-play
book -i inventory.ini web.yml
interpreter at /usr/bin/python3.7, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [3.29.137.251]

TASK [Install Apache] **********************************************************
ok: [3.29.137.251]

TASK [Start Apache] ************************************************************
ok: [3.29.137.251]

TASK [Create index.html] *******************************************************
changed: [3.29.137.251]

PLAY RECAP *********************************************************************
3.29.137.251               : ok=4    changed=1    unreachable=0    failed=0    skipp
ed=0    rescued=0    ignored=0

@mariahirfan658-ui ➜/workspaces/final-proj/Project6/terraform (main) $
```

```
 GNU nano 7.2                          web.yml
name: Configure web server
hosts: web
become: yes
tasks:
  - name: Install Apache
    yum:
      name: httpd
      state: present

  - name: Start Apache
    service:
      name: httpd
      state: started
      enabled: yes

  - name: Create index.html
    copy:
      dest: /var/www/html/index.html
                              [ Read 21 lines ]
```

```
GNU nano 7.2                    ansible/web-stack.yml *

  - name: Start services
    service:
      name: "{{ item }}"
      state: started
      enabled: yes
    loop:
      - nginx
      - php-fpm
```

```
GNU nano 7.2                          inventory.ini
[web]
3.29.137.251 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/proj-key-pai
```



---

8. Monitoring System

A custom monitoring script was deployed.

check-site.sh Functions

- Sends HTTP requests
- Measures response time
- Logs status codes

Automation

Runs every 5 minutes using cron:

*/5 * * * * /usr/local/bin/check-site.sh

This provides **basic uptime and latency monitoring**.

9. Customer Onboarding Procedure

To add a new website:

1. Add files in websites/customerX/
2. Run Ansible onboarding playbook
3. Verify via Load Balancer IP
4. SSL automatically applied

This ensures **consistent deployment for every customer**.

10. Performance Optimization Summary

Component	Optimization
Nginx LB	Caching, keepalive, upstream balancing
Nginx Web	Worker tuning, gzip
PHP-FPM	Dynamic process manager
Architecture	Multi-AZ redundancy

11. Challenges Faced

- Configuring SSL for multiple sites
- Ensuring Nginx upstream failover works
- Automating idempotent Ansible roles
- Managing private subnet connectivity

These were solved using **incremental testing and modular design**.

12. Conclusion

This project successfully demonstrates how to design and deploy a **cloud-based multi-tenant hosting platform** using open-source tools and AWS infrastructure.

By combining Terraform and Ansible, we achieved:

- Automation

- High availability
- Secure architecture
- Repeatable deployments

The platform can be extended with:

- Auto Scaling
- Managed databases
- Centralized logging
- Real SSL certificates (Let's Encrypt)

This project reflects real-world **DevOps and Cloud Engineering practices**.

****